

Week 2 Benchmark Report by Group 4

Methodology

The recorded execution environment is Apache Spark 4.0.1 running on local[*] with eight shuffle partitions. Because execution occurred on a local development workstation, these timings illustrate architectural and execution plan trade-offs rather than portable production targets.

As required by the assignment, every analytical query is compared before and after optimization across all five optimization strategies: Gold preaggregation, in-memory table caching, Adaptive Query Execution (AQE) with shuffle coalescing, explicit partition pruning, and broadcast hash joins.

The benchmark evaluates 48 distinct cases (6 analytical queries across 8 experimental configurations). Each case runs one warmup execution followed by five timed iterations, yielding 240 measured query executions. Timings measure the entire latency including SQL planning, stage execution, and driver result collection (`collect()`). Result correctness validation, physical plan extraction, Gold product materialization, and cache population are excluded from query timing. Every optimized run was verified for semantic equivalence against its baseline answer, asserting exact key matching, identical row counts and null distributions, and floating-point agreement within tolerance (`rel_tol=1e-9`, `abs_tol=1e-8`).

Benchmark Results

Full-period queries: Silver, Gold, caching and AQE

These comparisons use the same January-March answers. Silver calculates from detailed trips; Gold uses saved summaries. Caching stores the shared trips input, while AQE adjusts the execution plan at runtime. All values below are median seconds.

Query	Silver baseline	Gold	Cached trips	AQE enabled
Monthly zone demand	0.935	0.293	1.484	0.971
Distance by weather	0.565	0.274	0.514	0.543
Air quality and demand	1.189	0.228	0.504	1.116
Weather demand variation	4.150	0.375	5.426	3.330
Weekday peak hours	1.468	0.344	1.343	2.916
Monthly demand trends	1.627	0.400	1.190	2.827

Partition pruning: identical date windows

Silver is already partitioned by year and month. We compare a date filter with the same filter plus explicit year/month predicates.

Five queries use February, selected as the benchmark month. Monthly trends read January and February to calculate February's change. Both sides of each comparison use identical dates.

Query	Date filter (s)	Date + partition filters
Monthly zone demand	1.839	1.782
Distance by weather	1.257	1.328
Air quality and demand	0.730	0.770
Weather demand variation	2.232	2.371
Weekday peak hours	0.770	0.798
Monthly demand trends	0.845	0.812

Broadcast joins: rebuilding the enriched input

These tests join clean trips to the zone lookup and a small hourly weather/PM2.5 table. We build the hourly table by grouping environmental values already attached to integrated trips by pickup hour, then cache it for both variants.

The comparison is regular joins versus broadcast-hinted joins. These timings include enrichment joins that integrated Silver already stores, so they should be compared within this table.

Query	Without broadcast (s)	With broadcast
Monthly zone demand	3.857	1.250
Distance by weather	5.959	3.254
Air quality and demand	5.835	2.614
Weather demand variation	29.955	6.462
Weekday peak hours	3.003	1.371
Monthly demand trends	2.667	1.238

Performance Discussion

Millions of trips share relatively few zones and hourly environmental observations, making summaries compact and lookup tables suitable for broadcasting.

Gold: reuse completed calculations. Gold replaces detailed trip scans and repeated aggregations with small summary-table reads. The four products contain 714, 773, 786 and 2,183 rows. All queries improved by **2.06×** - **11.07×**; weather-demand variation benefited most because its zone totals and weather exposure were already calculated.

Caching: reuse input, but still do the analysis. The `EXPLAIN FORMATTED` plans contain `Scan In-memory table trips` and `InMemoryRelation`, confirming that the cache was used. Air-quality demand improved by **2.36×**, but monthly zone demand and weather-demand variation became slower. Aggregations and joins still run, and the original Parquet reads are already efficient. Cache access costs and memory competition are possible explanations. Caching smaller, reused intermediates is worth testing separately.

AQE: adjusting a plan also costs time. Plans show `AdaptiveSparkPlan`, `isFinalPlan=true` and coalesced `AQEShuffleRead` operators: Spark combined shuffle partitions during execution. Weather-demand variation improved by **1.25×**, while weekday peaks and monthly trends became substantially slower. With only eight partitions on one machine, adaptive planning overhead or reduced parallelism may outweigh savings. We did not measure stage-level costs, so the exact cause remains uncertain.

Partition pruning: applied, but little additional benefit. Before plans contain pushed-down date filters; after plans also contain explicit year/month `PartitionFilters`. Speedups were only **0.94×** - **1.04×**, with overlapping sample ranges. Delta file skipping and Parquet filtering may already avoid much of the unwanted data. We did not record files or bytes read, so we cannot claim both variants performed identical I/O.

Broadcasting: avoid moving the large input for lookup joins. Several `SortMergeJoin` operators become `BroadcastHashJoin` with `BroadcastExchange`. Distributing the small lookups avoids shuffling and sorting the large trip input for those joins. Other joins and aggregations can still shuffle. All six comparisons improved by **1.83×** - **4.64×**.

Preparation costs and limits

Preparation	Time	Trade-off
Build all four Gold products	23.40 s	Faster repeated queries, but storage and refresh work are required.
Populate the trips cache	10.85 s	Savings must repay preparation; the cache also uses memory.
Prepare the hourly broadcast lookup	2.47 s	Shared by both join variants and excluded from their timings.

Gold's active data files total **114,319 bytes**, excluding logs and old versions. Summing query medians gives about **9.93 seconds for Silver versus 1.91 seconds for Gold**. At that query mix, building Gold pays back after roughly three complete six-query rounds. This is an estimate, not a measured batch execution.